ADDRESSING MODES AND INSTRUCTION SET OF 8051

ADDRESSING MODES:

The CPU can access data from different places like registers, the memory or immediate values given by the programmer. The method (or channels) used by the CPU to access data to perform an operation is known as an addressing mode.

- **The 8051 provides 5 distinct addressing modes**

1. **Immediate Addressing Mode**
2. **Register Addressing Mode**
3. **Direct Addressing Mode**
4. **Register Indirect Addressing Mode**
5. **Indexed Addressing Mode**

Immediate Addressing Mode

In Immediate Addressing mode, the operand, which follows the Opcode, is a constant data of either 8 or 16 bits. The name Immediate Addressing came from the fact that the constant data to be stored in the memory immediately follows the Opcode.

The constant value to be stored is specified in the instruction itself rather than taking from a register. The destination register to which the constant data must be copied should be the same size as the operand mentioned in the instruction.

Example: MOV A, #030H

Here, the Accumulator is loaded with 30 (hexadecimal). The # in the operand indicates that it is a data and not the address of a Register.

Immediate Addressing is very fast as the data to be loaded is given in the instruction itself.

Register Addressing

In Register Addressing mode, one of the eight registers (R0 – R7) is specified as Operand in the Instruction. Source and destination registers must match in size.

It is important to select the appropriate Bank with the help of PSW Register. Let us see a example of Register Addressing assuming that Bank0 is selected.

Example: MOV A, R5

Here, the 8-bit content of the Register R5 of Bank0 is moved to the Accumulator.

Direct Addressing

In Direct Addressing Mode, the address of the data is specified as the Operand in the instruction. Using Direct Addressing Mode, we can access any register or on-chip variable. This includes general purpose RAM, SFRs, I/O Ports, Control registers.

Example: MOV A, 47H

Here, the data in the RAM location 47H is moved to the Accumulator.

Register Indirect Addressing

In the Indirect Addressing Mode or Register Indirect Addressing Mode, the address of the Operand is specified as the content of a Register. This will be clearer with an example.

Example: MOV A, @R1

The @ symbol indicates that the addressing mode is indirect. If the contents of R1 is 56H, for example, then the operand is in the internal RAM location 56H. If the contents of the RAM location 56H is 24H, then 24H is moved into accumulator.

Only R0 and R1 are allowed in Indirect Addressing Mode. These register in the indirect addressing mode are called as Pointer registers.

Indexed Addressing Mode

With Indexed Addressing Mode, the effective address of the Operand is the sum of a base register and an offset register. The Base Register can be either Data Pointer (DPTR) or Program Counter (PC) while the Offset register is the Accumulator (A).

In Indexed Addressing Mode, only MOVC and JMP instructions can be used. Indexed Addressing Mode is useful when retrieving data from look-up tables.

Example: MOVC A, @A+DPTR

Here, the address for the operand is the sum of contents of DPTR and Accumulator.

INSTRUCTION SET

- Instruction is command which we give to the microcontroller to perform any specific task
- Entire group of instructions is called instruction set. An 8051 Instruction consists of an Opcode (short of Operation – Code) followed by Operand(s) of size Zero Byte, One Byte or Two Bytes. The Op-Code part of the instruction contains the Mnemonic, which specifies the type of operation to be performed. All Mnemonics or the Opcode part of the instruction are of One Byte size.

Operand part of the instruction, defines the data being processed by the instructions. The operand can be any of the following:

- No Operand
- Data value
- I/O Port
- Memory Location
- CPU register

There can multiple operands and the format of instruction is as follows:

MNEMONIC DESTINATION OPERAND, SOURCE OPERAND

A simple instruction consists of just the opcode. Other instructions may include one or more operands. Instruction can be one-byte instruction, which contains only opcode, or two-byte instructions, where the second byte is the operand or three byte instructions, where the operand makes up the second and third byte.

- Types of instructions

1. Data Transfer Instruction
2. Arithmetic Instruction
3. Logical Instruction

4. Branching Instruction
5. Boolean Instruction

The Instruction Set and Addressing Modes

R_n	Register R7-R0 of the currently selected Register Bank.
direct	8-bit internal data location's address. This could be an Internal Data RAM location (0-127) or a SFR [i.e., I/O port, control register, status register, etc. (128-255)].
$@R_i$	8-bit internal data RAM location (0-255) addressed indirectly through register R1or R0.
#data	8-bit constant included in instruction.
#data 16	16-bit constant included in instruction.
addr 16	16-bit destination address. Used by LCALL and LJMP. A branch can be anywhere within the 64K byte Program Memory address space.
addr 11	11-bit destination address. Used by ACALL and AJMP. The branch will be within the same 2K byte page of program memory as the first byte of the following instruction.
rel	Signed (two's complement) 8-bit offset byte. Used by SJMP and all conditional jumps. Range is -128 to +127 bytes relative to first byte of the following instruction.
bit	Direct Addressed bit in Internal Data RAM or Special Function Register.

Data Transfer Instructions

Mnemonics	Description	Example	Byte	Machine Cycle
MOV A, Rn	Copy contents of Rn to A	MOV A,R4	1	1
MOV A, direct	Move content of direct address location to Accumulator	MOV A,56H	2	1
MOV A, @Ri	Copy the data that is at address in Ri to acc	MOV A,@R1	1	1
MOV A, #data	Copy the immediate data number into acc	MOV A,#05H	2	1
MOV Rn, A	Copy contents of A to Rn	MOV R3,A	1	1

MOV Rn, direct	Move content of direct address location to Rn	MOV R4, 45H	2	2
MOV Rn, #data	Copy the immediate data number in to Rn	MOV R6, #46H	2	1
MOV direct, A	Move content Acc to direct address location	MOV 89H, A	2	1
MOV direct, Rn	Move content of Rn to direct address location	MOV 65H, R5	2	2

Data transfer continued...

MOV direct, direct	Copy content of memory location in source to memory location in destination	MOV 68H, 45H	3	2
MOV direct, @Ri	Copy the data that is at address in Ri to direct memory location	MOV 54H, @R0	2	2
MOV direct, #data	Copy the immediate data to direct memory location	MOV 50H, #80H	3	2
MOV @Ri, A	Copy the data in Acc to the address in Ri	MOV @R1, A	1	1
MOV @Ri, direct	Copy the data from direct memory location to the address in Ri	MOV @R1, 45H	2	2
MOV @Ri, #data	Copy immediate data to the address in Ri	MOV @R0,#43H	2	1
MOV DPTR, #data16	Load data pointer with 16 bit constant		3	2

MOVC A, @A+DPTR	In this the address of instruction is formed by adding acc and DPTR and their content is copied to acc	-	1	2
MOVC A, @A+PC	In this the address of instruction is formed by adding acc and PC and its content is copied to acc	-	1	2
MOVX A, @Ri	Copy the content of external RAM 8 bit address in Ri to acc	MOVX A, @R0	1	2
MOVX A, @DPTR	Copy the content of external RAM 16 bit address in DPTR to acc	-	1	2
MOVX @Ri, A	Copy content of ACC to external RAM 8 bit address contained in Ri	MOVX @R0,A	1	2
MOVX @DPTR,A	Copy contents of acc to external RAM 16 bit address in DPTR	-	1	2
PUSH direct	In this instruction Increment Stack Pointer (SP) and copy the data from direct memory location to internal RAM address contained in SP	-	2	2
POP direct	This instruction copy the data from an internal RAM address contained in SP to direct memory location in the instruction and decrement SP	-	2	2
XCH A, Rn	Exchange the data between acc and Rn	XCH A, R1	1	1

XCH A, direct	Exchange the data between acc and direct memory location	XCH A, 45H	2	1
XCH A, @Ri	Exchange the data between acc and address in Ri	XCH A, @R1	1	1
XCHD A, @Ri	Exchange only lower nibble of acc and data at address in Ri	XCHD A, @R1	1	1

ARITHMETIC INSTRUCTIONS

Mnemonics	**Description**	**Byte**	Machine Cycle
ADD A, Rn	[A] ← [A]+[Rn]	1	1
ADD A,direct	[A] ← [A]+[Data at direct Address]	2	1
ADD A, @Ri	[A] ← [A]+[Data at Address pointed by Ri]	1	1
ADD A, #data	[A] ← [A]+[Data]	2	1
ADDC A, Rn	[A] ← [A]+[Rn]+[Carry flag]	1	1
ADDC A, direct	[A] ← [A]+[Data at direct Address]+[Carry flag]	2	1
ADDC A, @Ri	[A] ← [A]+[Data at Address pointed by Ri]+[Carry flag]	1	1
ADDC A, #data	[A] ← [A]+[Data]]+[Carry flag]	2	1
SUBB A, Rn	[A] ← [A]-[Rn]	1	1

SUBB A, direct	[A] [A]-[Data at direct Address]	2	1
SUBB A, @Ri	[A] [A]-[Data at Address pointed by Ri]	1	1
SUBB A, #data	[A] [A]-[Data]	2	1
INC A	[A] [A+1]	1	1
INC Rn	[Rn] [Rn+1]	1	1
INC direct	[Data at Address] [Data at Address+1]	2	1
DEC A	[A]<-[A-1]	1	1
DEC Rn	[Rn]<-[Rn-1]	1	1
DEC direct	Data at Address]<-[Data at Address-1]	2	1
DEC @Ri	[Data at Address pointed by register]<-[Data at Address pointed by register-1]	1	1
INC DPTR	DPTR]<-[DPTR +1]	1	2
MUL AB	[A]<-[A]*[B]	1	4
DIV AB	[A]<-[A]/[B]	1	4
DA A	Decimal adjust	1	1

LOGICAL INSTRUCTIONS

Mnemonics	Description	Byte	Machine Cycle
ANL A,Rn	[A]<-[A] AND [Rn]	1	1
ANL A, direct	[A]<-[A] AND [Data at Address]	2	1
ANL A, @Ri	[A]<-[A] AND [Data at Address in Ri]	1	1
ANL A, #data	[A]<-[A] AND [Data]	2	1
ANL direct, A	[Data at Address]<-[Data at Address] AND [A]	2	1
ANL direct, #data	[Data at Address]<-[Data at Address] AND [Data]	3	2
ORL A, Rn	[A]<-[A] OR [Rn]	1	1
ORL A, direct	[A]<-[A] OR [Data at Address]	2	1
ORL A, @Ri	A]<-[A] OR [Data at Address in Ri]	1	1
ORL A, #data	[A]<-[A] OR [Data]	2	1
ORL direct, A	[Data at Address]<-[Data at Address] OR [A]	2	1
ORL direct, #data	[Data at Address]<-[Data at Address] OR [Data]	3	2
XRL A, Rn	[A]<-[A] XOR [Rn]	1	1
XRL A, direct	[A]<-[A] XOR [Data at Address]	2	1
XRL A, @Ri	[A]<-[A] XOR [Data at Address in Ri]	1	1
XRL A, #data	[A]<-[A] XOR [Data]	2	1
XRL direct, A	[Data at Address]<-[Data at Address] XOR [A]	2	1
XRL direct, #data	[Data at Address]<-[Data at Address] XOR [Data]	3	2
CLR A	[A]<-0	1	1
CPL A	[A]<-[A]’	1	1

RL A	Shifts the value in accumulator to the left	1	1
RLC A	Shifts the value in accumulator to the left through the carry	1	1
RR A	Shifts the value in accumulator to the right	1	1
RRC A	Shifts the value in accumulator to the right through the carry	1	1
SWAP A	Swaps the upper nibble of the accumulator with the lower nibble	1	1

BRANCH INSTRUCTIONS

Mnemonics	**Description**	**Byte**	Machine Cycle
ACALL addr11	Absolute Call to subroutine in the maximum address range of 2K bytes(2^11=2K)	2	2
LCALL addr16	Long subroutine call in the maximum address range of 64K bytes(2^16=64K)	3	2
RET	Returns the control from subroutine	1	2
RETI	Returns the control from an interrupt subroutine	1	2
AJMP addr11	Absolute Jump to an address in a 2KB range	2	2
LJMP addr16	Long Jump to an address in a 64KB range	3	2
SJMP rel	Short Jump to an address in a 256-byte range (0 to 127 (0-7FH) range and -1 to -128 (FFH-80H).	2	2
JMP @A+DPTR	Unconditional jump to address [DPTR]<-[DPTR+A]	1	2

JZ rel	Jumps to address when accumulator=0	2	2
JNZ rel	Jumps to address when accumulator!=0	2	2
CJNE A, direct, rel	Compare accumulator contents with data at direct address and jump to relative address if they are not equal	3	2
CJNE A, #data, rel	Compare accumulator contents with immediate data and jump to relative address if they are not equal	3	2
CJNE Rn, # data, rel	Compare register contents with immediate data and jump to relative address if they are not equal	3	2
CJNE @Ri, #data, rel	Compare data at address pointed by Ri with immediate data and jump to relative address if they are not equal	3	2
DJNZ Rn, rel	Decrements value in Rn and jump to relative address till Rn!=0	2	2
DJNZ direct, rel	Decrement the direct byte and jump to relative address if not zero	3	2
NOP	No operation	1	1

BOOLEAN INSTRUCTIONS

Mnemonics	Description	Byte	Machine Cycle
CLR C	Clear carry	1	1
CLR bit	Clear direct bit	2	1
SETB C	Set carry	1	1
SETB bit	Set direct bit	2	1
CPL C	Complement carry	1	1
CPL bit	Complement bit	2	1
ANL C, bit	AND direct bit to carry	2	2
ANL C, /bit	AND complement of direct bit to carry	2	2
ORL C, bit	OR direct bit to carry	2	2
ORL C, /bit	OR complement of direct bit to carry	2	2
MOV C ,bit	Move direct bit to carry	2	1
MOV bit, C	Move carry to direct bit	2	2
JC rel	Jump if carry is set	2	2
JNC rel	Jump if carry not set	2	2
JB bit, rel	Jump if direct bit is set	3	2
JNB bit, rel	Jump if direct bit is not set	3	2
JBC bit, rel	Jump if direct bit is set and clear bit	3	2